

Research Agenda, Elaborated: Implementation of Each Step

This document expands each of the six steps of the research agenda into a concrete implementation plan: what is produced, which tools are used, and how the step is actually run.

Step 1: Human baseline

The purpose of this step is to obtain a ground-truth process trace against which the autoresearch loop can later be compared. Without it, any statement about the loop’s behavior has nothing to be judged against.

To implement it, I pick one concrete sub-problem within the random forest / gradient boosting domain, phrased exactly as it would be phrased to the autoresearch loop, for example ”does the feature subsampling ratio affect generalization of a random forest on dataset X”. I then work through this problem myself while recording screen and voice (a simple tool like OBS is sufficient), thinking aloud as I go: what I hypothesize, what experiment I design to test it, what I observe, and what I decide to do next. The recording is transcribed automatically (e.g. with Whisper) and then manually segmented into discrete action units using a small, fixed taxonomy: hypothesize, design experiment, execute, observe, interpret, decide next step. Each unit is a tuple of (action type, content, timestamp). The result is an ordered trace $\tau_{human} = (a_1, a_2, \dots, a_n)$ that plays the same role as a reference solution in supervised learning: everything downstream is compared to it.

If time allows, a second person performs the same task independently, which allows a basic inter-rater check on how stable the ”human” reference itself is; this is optional and only useful if the first trace looks idiosyncratic.

Step 2: Program.md as a configuration space

The purpose of this step is to turn an unstructured text file into a small number of variables that can be manipulated and measured, the same way a hyperparameter vector turns an algorithm’s behavior into something that can be studied systematically.

To implement it, I first read through several existing program.md files and existing autoresearch transcripts and list every instructional element that plausibly affects behavior: whether an evaluation metric is specified, how broad or narrow the exploration instruction is, whether a stopping criterion is given, whether the output format is prescribed, whether examples are included, and the order in which instructions appear. From this list I select a small number of axes, typically five to six, each with two or three discrete levels (e.g. ”evaluation metric: unspecified / accuracy only / accuracy plus variance”). This gives a configuration vector $c = (c_1, \dots, c_k)$, where each c_i is categorical.

Practically, I do not hand-write each program.md variant. Instead I write one template with placeholders (a simple string or Jinja2 template) and a small script that, given a configuration vector, fills in the placeholders and writes out the corresponding program.md file. This guarantees that every variant differs from every other variant only in the axes I intended to vary, which is essential for anything I want to interpret causally later.

Step 3: Ablation instead of random sampling

The purpose of this step is to decide which configurations to actually run. With five to six axes at two to three levels each, a full factorial design already means 32 to hundreds of runs, and each run may need

to be repeated several times because the loop itself is stochastic (temperature, sampling). Running all of them is usually not affordable, so the design of experiments has to be chosen deliberately rather than by convenience or by random sampling.

To implement it, I use a fractional factorial or Plackett-Burman design, generated with a standard design-of-experiments library such as `pyDOE2`, which selects a subset of configurations that still allows the main effect of each axis (and, if enough runs are affordable, the two-way interactions) to be estimated. This is the same design principle used in AutoML ablation studies that trace performance differences back to individual configuration changes (e.g. Fawcett and Hoos on ablation analysis for algorithm configuration). For each selected configuration I generate the corresponding `program.md` with the script from step 2, then run the autoresearch loop three to five times per configuration with different random seeds, to separate the effect of the configuration itself from the effect of the loop’s inherent randomness. Every run’s full transcript, generated code, and final report are saved to a directory named by configuration id and repeat id, so that nothing has to be re-run later to recover what happened.

Step 4: Behavioral encoding

The purpose of this step is to turn each run’s raw transcript into the same kind of representation as the human baseline trace, so that the two are comparable, and to additionally summarize each run as a small number of numbers that can be analyzed statistically.

To implement it, I build a parser that walks each run’s transcript and log files and segments them into the same action taxonomy used in step 1 (hypothesize, design, execute, observe, interpret, decide). If the autoresearch loop already produces structured logs (e.g. JSON entries per loop iteration), this segmentation is largely mechanical; if it only produces free text, a language model can be used to classify each transcript segment into one of the taxonomy categories, which then has to be spot-checked by hand for reliability. From the resulting trace τ_i for run i , I compute a small set of behavioral features: the number of distinct algorithmic variants explored (e.g. distinct hyperparameters or ensemble variants mentioned), a process-similarity score between τ_i and τ_{human} (e.g. via edit distance or dynamic time warping over the action sequence), and product-level features such as the final validation score achieved on a fixed held-out dataset, or the number of experiments actually executed versus merely proposed. The output of this step is a single table with one row per run: the configuration vector c , the repeat id, and the behavioral features, ready for the next step.

Step 5: Surrogate model (the white box)

The purpose of this step is to fit a small, interpretable model that explains how the configuration vector c determines the behavioral features, which is the actual white-box artifact this project is meant to produce.

To implement it, for each behavioral feature y I fit a model $y \approx f(c)$ using the table from step 4. Given that the number of runs is likely small (dozens rather than thousands), I prefer models that are easy to interpret with little data: a shallow decision tree of depth two or three, or a linear regression with pairwise interaction terms between configuration axes. Where enough runs are available, functional ANOVA (as implemented in the `fanova` package originally built for hyperparameter importance analysis in SMAC) can be used to decompose the variance of y into the fraction attributable to each axis individually and to low-order interactions between axes, which gives a directly interpretable statement of the form “axis c_3 (stopping criterion) explains 40% of the variance in search breadth”. This step is what actually answers the professor’s underlying question: instead of only knowing that changing `program.md` changes the output, the surrogate model states which part of `program.md` changes which part of the behavior, and by how much.

Step 6: Causal validation

The purpose of this final step is to check whether the surrogate model from step 5 is actually predictive, and not merely a description of the data it was fit on. A model that only correlates is still, in an important sense, a black box; a model that lets me intervene and get the predicted result is a white box.

To implement it, I pick a target value for one behavioral feature that was not directly observed in the ablation design, and use the surrogate model to search over the remaining, not-yet-run configurations for the one predicted to produce that value (a simple grid search over the discrete configuration space is sufficient here, since k is small). I generate the corresponding `program.md`, run the autoresearch loop several times on it, and compare the observed behavioral feature to the model's prediction. If the prediction is close, this is direct evidence that the causal pathway from `program.md` to behavior, as captured by the surrogate, is correct rather than coincidental; if it is not close, the discrepancy itself is informative about which axes or interactions were missed in step 2 or step 5, and the agenda loops back to refining the configuration space rather than being a dead end.